

Conceitos de Criptografia

senhasegura

Departamento de Segurança e Inovação - SEGi9



PÚBLICO



Resumo Executivo

Este texto tem como objetivo descrever os principais conceitos da área de segurança, com foco especial na área de criptografia.





Sumário

1	Princípios	1
2	Criptografia do Lado do Cliente	2
2.1	Funcionamento da Criptografia do Lado do Cliente	2
3	Confidencialidade	3
3.1	Algoritmo Simétrico	3
3.2	Algoritmo Assimétrico	5
3.3	Key-Encryption-Key (KEK)	8
3.4	Key Derivation Functions (KDF)	11
4	Integridade	14
4.1	Hash	14
5	Autenticidade	16
5.1	Assinatura Digital	16
6	Confidencialidade com Criptografia Híbrida	19





1 Princípios

A segurança da informação é um campo fundamental no mundo digital atual, onde a proteção dos dados e informações é essencial para a manutenção da privacidade e a confiabilidade das operações.

Neste documento, vamos abordar três objetivos de segurança da informação: confidencialidade, integridade e autenticidade.

- Confidencialidade
- Integridade
- Autenticidade

A criptografia do lado do cliente, descrita na seção seguinte e por vezes chamada de arquitetura de conhecimento zero, é um modelo de implementação, não um quarto objetivo nem uma prova criptográfica de conhecimento zero.





2 Criptografia do Lado do Cliente

Neste contexto, **criptografia do lado do cliente** (também chamada de arquitetura de conhecimento zero) significa que os dados são cifrados e decifrados no dispositivo do usuário, e o serviço é projetado para não receber o texto plano, a Senha Mestra ou as chaves de dados em claro. O termo não se refere a uma prova criptográfica de conhecimento zero, que é uma técnica distinta para provar uma afirmação sem revelar o segredo usado na prova.

2.1 Funcionamento da Criptografia do Lado do Cliente

1. Derivação da chave

- O cliente combina a Senha Mestra com um salt aleatório e usa uma KDF apropriada para derivar uma chave. O salt e os parâmetros da KDF podem ser armazenados junto aos dados cifrados; eles não precisam ser secretos.

2. Cifragem local

- O cliente cifra as credenciais e outros dados localmente antes de enviá-los ao serviço.
- Deve-se usar uma cifra autenticada (AEAD), como AES-GCM, com um nonce novo e único para cada operação sob a mesma chave. O texto cifrado, o nonce e a tag de autenticação são enviados e armazenados juntos.

3. Armazenamento no serviço

- O serviço armazena os dados cifrados e os metadados necessários para derivar a chave e decifrá-los, mas não deve receber as chaves em claro.
- O serviço ainda pode conhecer metadados não cifrados, como tamanhos, horários e informações da conta, e pode apagar, reter ou reverter dados armazenados.

4. Recuperação e decifragem

- O cliente baixa os dados cifrados, deriva novamente a chave com a Senha Mestra e o salt armazenado e verifica a tag AEAD antes de liberar o texto plano.
- Se a verificação falhar, a chave ou os parâmetros podem estar incorretos, ou os dados podem ter sido alterados; a falha, por si só, não identifica a causa.

Esse modelo reduz o acesso do provedor ao conteúdo, mas não torna os dados automaticamente seguros diante de qualquer comprometimento. Uma senha mestra fraca pode permitir tentativas de adivinhação offline contra os dados cifrados; um dispositivo comprometido pode capturar a senha ou o texto plano. Sem uma chave de recuperação independente, perder a Senha Mestra pode tornar os dados irrecuperáveis.





3 Confidencialidade

A Confidencialidade refere-se à garantia de que as informações são acessíveis apenas por pessoas autorizadas. Em outras palavras, é a proteção contra a divulgação não autorizada de dados. Este pilar é essencial para manter a privacidade dos dados pessoais e sensíveis, como informações financeiras, médicas ou estratégicas de uma organização. Técnicas de criptografia são amplamente utilizadas para garantir a confidencialidade, assegurando que, mesmo que os dados sejam interceptados, eles não possam ser lidos ou compreendidos por indivíduos não autorizados.

3.1 Algoritmo Simétrico

Entre os diversos tipos de algoritmos de criptografia, o algoritmo de criptografia simétrica é um dos mais comuns e amplamente utilizados. Neste tipo de algoritmo, a mesma chave(K) é utilizada tanto para criptografar(C), gerando a mensagem cifrada(M), quanto para descriptografar(D) os dados, gerando o Texto Plano(T).

A Figura 1 ilustra o funcionamento do algoritmo de criptografia simétrica. Para mostrar o funcionamento do algoritmo, vamos considerar três personagens: **Alice**, **Bob** e **Eva**. Alice e Bob desejam se comunicar de forma segura, enquanto Eva deseja ler a mensagem trocadas entre eles.

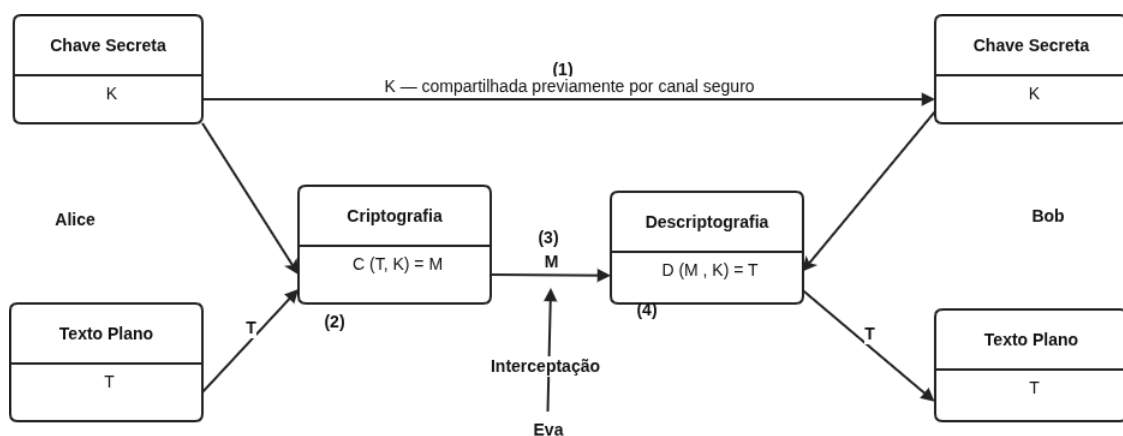


Figura 1: Funcionamento do algoritmo de criptografia simétrico





Funcionamento do Algoritmo de Criptografia Simétrico

1. Alice e Bob precisam obter a mesma chave secreta (K) por um mecanismo seguro antes de trocar mensagens. O comando abaixo gera uma chave aleatória de 256 bits para o exemplo; executá-lo em Alice não entrega automaticamente uma cópia a Bob.

```
$ umask 077  
$ openssl rand -hex 32 > K.hex
```

2. Quando Alice deseja enviar o Texto Plano(T) para Bob, ela utiliza AES-256-CBC, a Chave Secreta(K) e um IV aleatório para gerar o Texto Cifrado(M). O IV não é secreto e deve acompanhar o texto cifrado; gere um IV novo e imprevisível para cada cifragem. Este exemplo demonstra apenas confidencialidade: em produção, prefira uma cifra autenticada (AEAD), como AES-GCM, ou combine CBC com um MAC usando uma chave independente e autentique o IV e o texto cifrado antes de decifrar.

```
$ IV_HEX=$(openssl rand -hex 16)  
$ printf '%s' "$IV_HEX" > M.iv  
$ openssl enc -aes-256-cbc -K "$(cat K.hex)" -iv "$IV_HEX" \  
  -in T.txt -out M.enc
```

3. Alice envia o Texto Cifrado(M) e o IV para Bob através de um canal de comunicação inseguro. Se Eva apenas interceptar esses dados e a chave permanecer secreta, ela não deverá conseguir recuperar a mensagem original. Como CBC sozinho não autentica os dados, um atacante ativo ainda pode alterar o texto cifrado ou o IV sem que a alteração seja detectada de forma confiável.
4. Ao receber o Texto Cifrado(M), Bob utiliza o mesmo algoritmo de Descifragem(D) e a Chave Secreta(K) para transformar o Texto Cifrado(M) de volta ao Texto Plano(T).

```
$ openssl enc -d -aes-256-cbc -K "$(cat K.hex)" -iv "$(cat M.iv)" \  
  -in M.enc -out T.txt
```

A principal vantagem dos algoritmos de criptografia simétrica é sua eficiência e velocidade, especialmente em comparação com os algoritmos de criptografia assimétrica. No entanto, Alice e Bob precisam estabelecer a chave secreta com segurança. Se Eva descobrir essa chave, poderá decifrar as mensagens protegidas por ela.

Esses comandos são didáticos: o arquivo K.hex contém a chave em claro, e a opção -K passa a chave como argumento do processo, onde ela pode ficar visível durante a execução. Em produção, use uma biblioteca criptográfica e um mecanismo de armazenamento e uso de chaves





que evitem essa exposição. Alice e Bob podem provisionar a chave por um canal protegido ou estabelecê-la por um protocolo autenticado de acordo de chaves.

3.2 Algoritmo Assimétrico

O algoritmo de criptografia assimétrica é uma técnica fundamental no campo da segurança e criptografia. Diferente dos algoritmos de criptografia simétrica, que utilizam a mesma chave para criptografar e descriptografar os dados, os algoritmos assimétricos usam um par de chaves: uma chave pública(KPu) e uma chave privada(KPr). Vamos explorar o funcionamento desse algoritmo através dos personagens Alice, Bob e Eva exemplificado pela Figura [2](#).



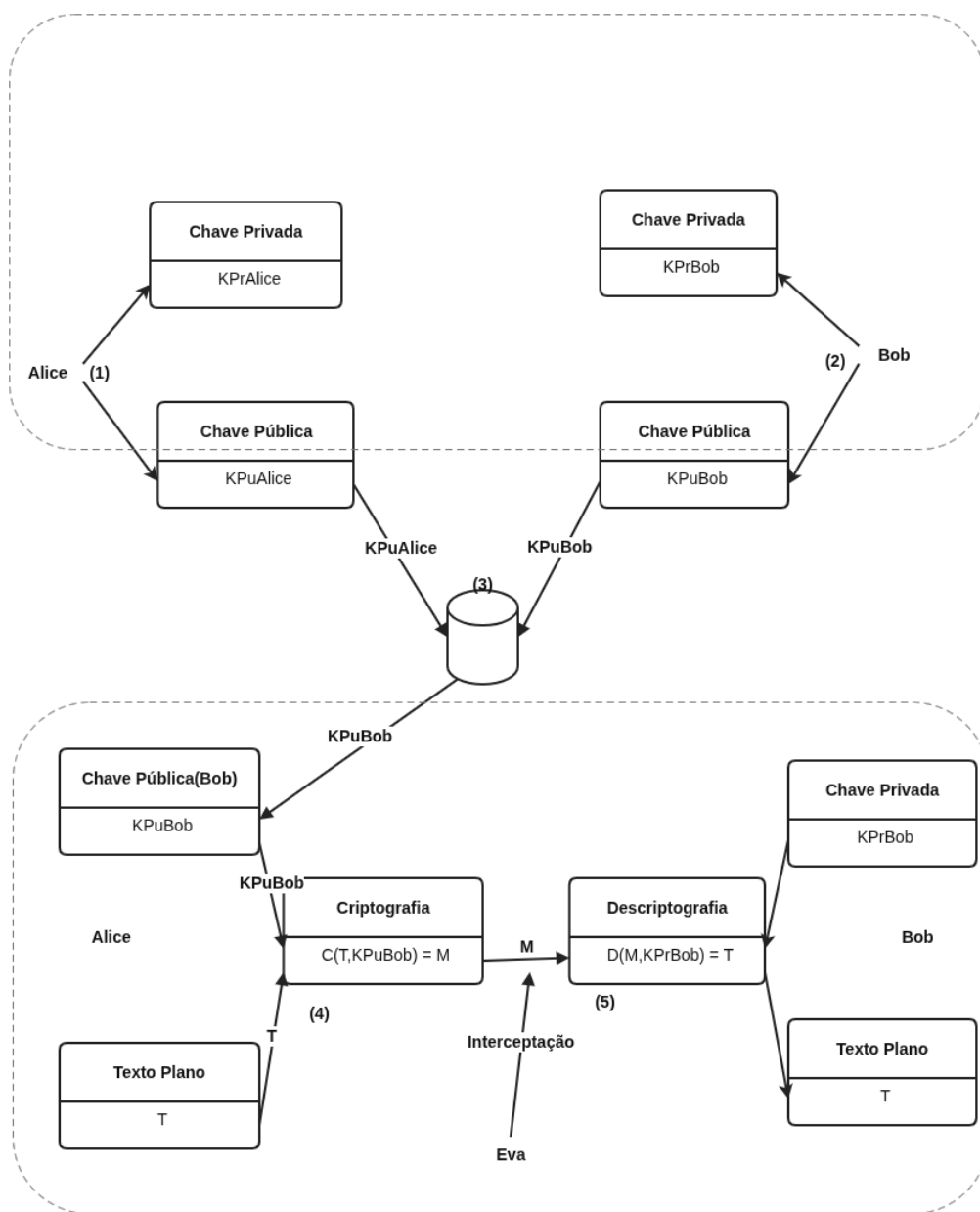


Figura 2: Funcionamento do algoritmo de criptografia assimétrico





Funcionamento do Algoritmo de Criptografia Assimétrico

1. Alice gera um par de chaves, Chave Pública(KPuAlice) e Chave Privada(KPrAlice);

```
$ openssl genpkey -algorithm RSA -out KPrAlice.pem -pkeyopt rsa_keygen_bits:2048  
$ openssl rsa -pubout -in KPrAlice.pem -out KPuAlice.pem
```

2. Bob gera um par de chaves, Chave Pública(KPuBob) e Chave Privada(KPrBob);

```
$ openssl genpkey -algorithm RSA -out KPrBob.pem -pkeyopt rsa_keygen_bits:2048  
$ openssl rsa -pubout -in KPrBob.pem -out KPuBob.pem
```

3. Tanto Alice quanto Bob enviam as suas respectivas chaves públicas, KPuAlice e KPuBob, para um servidor de chave pública;
4. Antes de usar a chave pública recuperada do servidor, Alice deve verificar por um meio confiável que ela realmente pertence a Bob, por exemplo, validando um certificado digital ou comparando sua impressão digital por um canal autenticado. A presença da chave no servidor, por si só, não comprova essa associação: um atacante que substitua a chave de Bob pela sua própria pode interceptar a comunicação;
5. Quando Alice deseja enviar um Texto Plano (T) curto para Bob, ela utiliza a Chave Pública do Bob (KPuBob.pem) para gerar o Texto Cifrado (M). Com RSA de 2048 bits e OAEP/SHA-256, a entrada deve ter no máximo 190 bytes. Para mensagens maiores, usa-se criptografia híbrida: Alice cifra a mensagem com uma chave simétrica aleatória usando uma cifra autenticada, como AES-GCM, e cifra essa chave simétrica com a chave pública de Bob.

```
$ openssl pkeyutl -encrypt -inkey KPuBob.pem -pubin -in T.txt -out M.bin \  
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

6. Ao receber o Texto Cifrado(M), o Bob utiliza sua Chave Privada(KPrBob) para descifrá-la obtendo o Texto Plano(T).

```
$ openssl pkeyutl -decrypt -inkey KPrBob.pem -in M.bin -out T.txt \  
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

Se Alice usar uma chave pública de Bob cuja associação com ele foi verificada, e Bob proteger sua chave privada, a cifragem com RSA-OAEP protege a confidencialidade contra Eva, mesmo que ela intercepte o texto cifrado. Esse fluxo, por si só, não comprova que a mensagem foi enviada por Alice: qualquer pessoa que conheça a chave pública de Bob pode cifrar mensagens para





ele. Para autenticar a origem, Alice deve assinar a mensagem com sua chave privada, e Bob deve verificar a assinatura usando uma chave pública de Alice cuja associação também seja confiável.

Na criptografia híbrida, a cifra autenticada permite detectar alterações no conteúdo cifrado, mas não identifica quem o produziu. A assinatura digital fornece essa verificação de origem, desde que a chave pública do remetente esteja autenticada.

3.3 Key-Encryption-Key (KEK)

Uma Key-Encryption Key (KEK), ou chave de criptografia de chaves, é usada para proteger (encapsular) outras chaves. Neste exemplo, MKBob, derivada da Senha Mestra por uma KDF, exerce o papel de KEK: ela protege a DEK, que por sua vez cifra os dados. Essa separação facilita o gerenciamento e a troca de chaves, mas não garante segurança por si só.

A Figura 3 ilustra a hierarquia de chaves: M1 é a DEK protegida pela KEK, e M2 são os dados protegidos pela DEK.



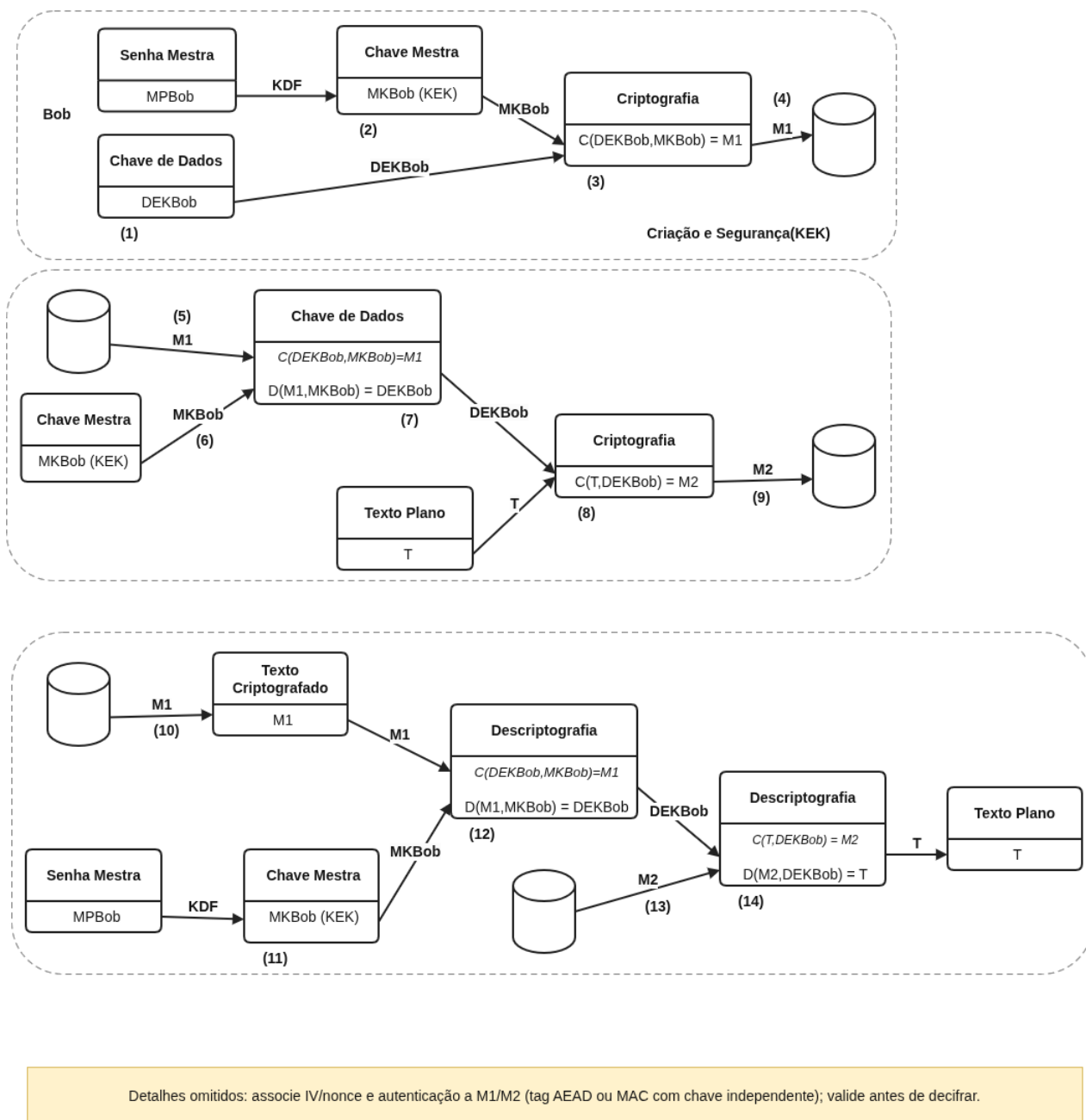


Figura 3: Funcionamento do conceito Key-Encryption-Key (KEK)

Nota sobre os comandos: são exemplos didáticos e usam AES-CBC sem autenticação. Em produção, prefira AEAD para proteger M1 e M2; se CBC for necessário, use um MAC com chave independente que cubra o IV e o texto cifrado, e verifique-o antes de decifrar. Armazene com M1 seu IV, o salt e os parâmetros da KDF; armazene com M2 seu IV. A cópia de M1 permite





tentativas offline de adivinhar a Senha Mestra, portanto a KDF não substitui uma senha forte. Os exemplos são didáticos e gravam material secreto em arquivos locais. O comando ‘openssl kdf’ recebe a Senha Mestra como argumento de processo, e os comandos AES-CBC também passam a chave dessa forma; esses exemplos não devem ser usados assim em produção.

- Criptografar a Chave de Dados(DEKBob)

1. Bob gera uma DEK aleatória de 256 bits;

```
$ set -o pipefail
$ umask 077
$ openssl rand -out DEKBob.bin 32
```

2. Bob deriva a KEK (MKBob.hex) a partir da Senha Mestra com PBKDF2-HMAC-SHA-256, 600.000 iterações e um salt aleatório de 16 bytes (128 bits). O salt e os parâmetros da KDF não são secretos e devem ser preservados para a recuperação;

```
$ umask 077
$ set -o pipefail
$ if [ ! -s MKBob.salt ]; then openssl rand -hex 16 > MKBob.salt; fi
$ SALT_HEX=$(cat MKBob.salt)
$ read -rs MASTER_PASSWORD; printf '\n'
$ printf 'PBKDF2-HMAC-SHA-256\niterations=600000\nkey_length=32\n' > MKBob.kdf
$ openssl kdf -keylen 32 -kdfopt digest:SHA256 \
-kdfopt "pass:$MASTER_PASSWORD" -kdfopt "salt:$SALT_HEX" \
-kdfopt iter:600000 PBKDF2 | tr -d ':' > MKBob.hex.tmp \
&& mv MKBob.hex.tmp MKBob.hex
$ chmod 600 MKBob.salt MKBob.hex MKBob.kdf
$ unset MASTER_PASSWORD
```

3. Bob protege (encapsula) a DEK com a KEK MKBob, gerando M1. O IV acompanha M1;

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M1.iv
$ openssl enc -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$IV_HEX" \
-in DEKBob.bin -out M1.enc
```

4. Bob envia M1, seu IV, o salt e os parâmetros da KDF para o servidor;

- Criptografar o Texto Plano(T) com uma Chave de Dados DEKBob

5. Bob recupera o Texto Cifrado M1 do servidor;
6. Bob possui sua Chave Mestra (MKBob.hex);
7. Bob descriptografa o Texto Cifrado M1 para obter acesso à sua Chave de Dados (DEKBob.bin);





```
$ openssl enc -d -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$(cat M1.iv)" \
-in M1.enc -out DEKBob.bin
```

8. Bob cifra o Texto Plano (T) com a DEK e um IV aleatório, gerando M2. O IV acompanha M2;

```
$ DEK_HEX=$(od -An -v -tx1 DEKBob.bin | tr -d ' \n')
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M2.iv
$ openssl enc -aes-256-cbc -K "$DEK_HEX" -iv "$IV_HEX" \
-in T.txt -out M2.enc
```

9. Bob envia M2 e seu IV para o servidor;

- Descriptografar o Texto Cifrado M2 para obter o Texto Plano(T)

10. Bob recupera o Texto Cifrado M1 do servidor;
11. Bob possui sua Chave Mestra (MKBob.hex);
12. Bob descriptografa o Texto Cifrado M1 com sua Chave Mestra (MKBob.hex) para obter acesso à sua Chave de Dados (DEKBob.bin);

```
$ openssl enc -d -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$(cat M1.iv)" \
-in M1.enc -out DEKBob.bin
```

13. Bob recupera o Texto Cifrado M2 do servidor;
14. Bob descriptografa o Texto Cifrado M2 com sua Chave de Dados (DEKBob.bin) para obter acesso ao Texto Plano (T);

```
$ DEK_HEX=$(od -An -v -tx1 DEKBob.bin | tr -d ' \n')
$ openssl enc -d -aes-256-cbc -K "$DEK_HEX" -iv "$(cat M2.iv)" \
-in M2.enc -out T.txt
```

Uma KEK protege chaves de dados e ajuda a separar a gestão de chaves da cifragem dos dados. A confidencialidade e a detecção de alterações dependem também dos algoritmos, modos e metadados usados; a hierarquia KEK/DEK, sozinha, não garante integridade nem autenticidade.

3.4 Key Derivation Functions (KDF)

As funções de derivação de chaves (Key Derivation Functions – KDFs) transformam um segredo inicial, como uma senha, em uma ou mais chaves criptográficas de tamanho definido. A Figura





4 ilustra a derivação de uma chave a partir de uma Senha Mestra. A derivação também depende de um salt e de parâmetros que controlam o custo da operação.

O salt deve ser único para cada derivação e pode ser armazenado junto aos dados e aos parâmetros da KDF; ele não precisa ser secreto. Seu uso evita que senhas iguais produzam necessariamente a mesma chave e dificulta o pré-cálculo de tabelas. Uma KDF apropriada para senhas aumenta o custo de cada tentativa de adivinhação, mas não elimina ataques de força bruta nem compensa uma senha fraca.

Um exemplo é PBKDF2 (Password-Based Key Derivation Function 2), especificada no PKCS #5. Ela aplica uma função pseudorrandômica, como HMAC-SHA-256, repetidamente; o número de iterações é um de seus parâmetros de custo. Argon2, selecionada na Password Hashing Competition, é outra opção e permite configurar parâmetros como memória, tempo e paralelismo. Os parâmetros utilizados devem ser armazenados para possibilitar a derivação posterior da mesma chave e escolhidos conforme as recomendações atuais e os recursos disponíveis.

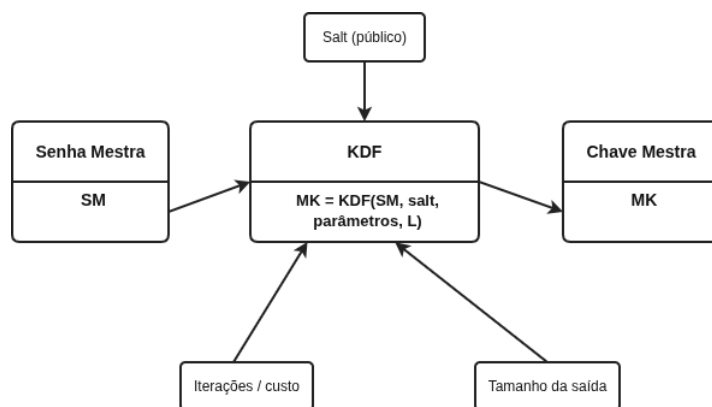


Figura 4: Derivação de chave a partir de uma senha, salt e parâmetros da KDF

Os comandos a seguir usam o OpenSSL para derivar uma chave com PBKDF2-HMAC-SHA-256. Na primeira execução, geram e salvam um salt aleatório; nas seguintes, reutilizam o salt salvo para derivar novamente a mesma chave. O arquivo MKBob.kdf registra algoritmo, iterações e tamanho da chave.

1. Derivar uma chave de 256 bits usando PBKDF2 e salvar sua representação hexadecimal localmente. O salt é armazenado junto dos metadados necessários à derivação.





```
$ umask 077
$ set -o pipefail
$ if [ ! -s MKBob.salt ]; then openssl rand -hex 16 > MKBob.salt; fi
$ SALT_HEX=$(cat MKBob.salt)
$ read -rs MASTER_PASSWORD; printf '\n'
$ printf 'PBKDF2-HMAC-SHA-256\niterations=600000\nkey_length=32\n' > MKBob.kdf
$ openssl kdf -keylen 32 -kdfopt digest:SHA256 \
  -kdfopt "pass:$MASTER_PASSWORD" -kdfopt "salt:$SALT_HEX" \
  -kdfopt iter:600000 PBKDF2 | tr -d ':\n' > MKBob.hex.tmp \
  && mv MKBob.hex.tmp MKBob.hex
$ chmod 600 MKBob.salt MKBob.hex MKBob.kdf
$ unset MASTER_PASSWORD
```





4 Integridade

A integridade trata da detecção de alterações indevidas nos dados. Funções de hash, MACs e assinaturas digitais podem apoiar essa verificação, mas oferecem garantias diferentes: um hash simples não autentica a origem, e seu uso para detectar adulteração deliberada depende de o resumo ser obtido por um meio confiável.

4.1 Hash

Um hash é uma função que transforma uma entrada de dados (que pode variar de alguns bytes a grandes volumes de dados) em uma saída de tamanho fixo, geralmente representada como uma sequência de caracteres alfanuméricos. Esta transformação é determinística, o que significa que a mesma entrada sempre produzirá a mesma saída.

Alguns exemplos de algoritmos hash populares incluem a família de algoritmos SHA-2 (Secure Hash Algorithm 2), que inclui SHA-224, SHA-256, SHA-384 e SHA-512. Os algoritmos SHA-2 produzem hashes de diferentes tamanhos, sendo o SHA-256 e o SHA-512 os mais comuns, gerando valores de 256 e 512 bits, respectivamente.

A Figura 5 mostra Bob enviando a mensagem e seu resumo para Alice. Ela calcula o resumo da mensagem recebida e compara os dois valores.

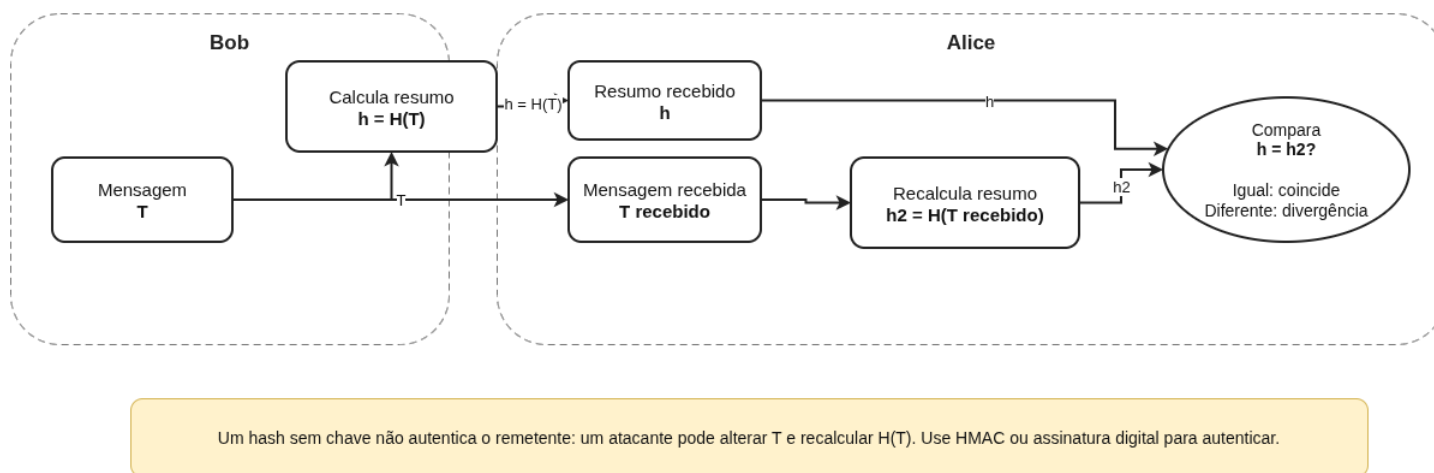


Figura 5: Comparação de resumos para detectar divergências em uma mensagem

Bob calcula $h = H(T)$ e envia a mensagem T junto com o resumo h . Alice calcula $h_2 = H(T_{\text{recebido}})$ sobre a mensagem recebida e compara h_2 com h . Se forem diferentes, a mensagem recebida diverge daquela que corresponde ao resumo. Se forem iguais, isso apenas indica que a mensagem coincide com o resumo recebido; um hash simples enviado junto com a mensagem não impede que um atacante altere ambos e recalcule o resumo. Para autenticar a origem e





proteger contra adulteração deliberada, use um HMAC com segredo compartilhado ou uma assinatura digital cuja chave pública de Bob seja autenticada. O hash não cifra a mensagem.

Prático

O OpenSSL calcula resumos SHA-256 e SHA-512. Para comparar um arquivo com um resumo salvo, obtenha esse resumo por um meio confiável: se um atacante puder substituir tanto o arquivo quanto o resumo, a comparação não detectará a adulteração.

1. Salvar o resumo SHA-256 de `arquivo.txt`. O resumo de referência precisa vir de uma fonte confiável.

```
$ openssl dgst -sha256 arquivo.txt | awk '{print $NF}' > hash.sha256
```

2. Comparar o resumo atual de `arquivo.txt` com o resumo SHA-256 salvo.

```
$ openssl dgst -sha256 arquivo.txt | awk '{print $NF}' | \
cmp -s - hash.sha256 && echo "Resumo igual" || \
{ echo "Resumo divergente" >&2; exit 1; }
```

3. Salvar o resumo SHA-512 de `arquivo.txt`. O resumo de referência precisa vir de uma fonte confiável.

```
$ openssl dgst -sha512 arquivo.txt | awk '{print $NF}' > hash.sha512
```

4. Comparar o resumo atual de `arquivo.txt` com o resumo SHA-512 salvo.

```
$ openssl dgst -sha512 arquivo.txt | awk '{print $NF}' | \
cmp -s - hash.sha512 && echo "Resumo igual" || \
{ echo "Resumo divergente" >&2; exit 1; }
```





5 Autenticidade

A Autenticidade está relacionada à verificação da identidade das partes envolvidas na comunicação e à garantia de que a origem das informações é confiável. Este pilar garante que as entidades (pessoas, sistemas ou dispositivos) sejam quem dizem ser e que as informações provenientes dessas entidades sejam genuínas. Métodos de autenticação, como senhas, certificados digitais e autenticação de dois fatores, são utilizados para validar a identidade e estabelecer a confiança entre as partes. A autenticidade é crucial para prevenir fraudes e assegurar a legitimidade das transações eletrônicas.

5.1 Assinatura Digital

Uma assinatura digital permite verificar se uma mensagem corresponde a uma assinatura produzida com a chave privada associada à chave pública usada na verificação. Ela ajuda a detectar alterações na mensagem e a autenticar a origem da assinatura, desde que a chave pública esteja autenticada. A assinatura digital não cifra a mensagem nem fornece confidencialidade.

Funcionamento

Bob calcula uma assinatura $\sigma = \text{Sign}(KPrBob, T)$ sobre a mensagem T e envia a mensagem junto com σ . Na prática, algoritmos de assinatura costumam aplicar internamente uma função de hash; isso não significa que o resumo seja simplesmente cifrado com a chave privada. Alice verifica a assinatura com $\text{Verify}(KPubBob, T, \sigma)$, que retorna válida ou inválida.

A Figura 6 mostra esse fluxo. Alice precisa obter uma chave pública que esteja de fato associada a Bob, por exemplo validando a cadeia de certificados ou conferindo o fingerprint por um canal independente. Sem essa validação, um atacante poderia substituir a chave pública por outra e fazer Alice aceitar uma assinatura sob a chave errada. Uma verificação válida confirma a correspondência entre mensagem, assinatura e chave pública; a atribuição da identidade de Bob depende da confiança nessa chave.



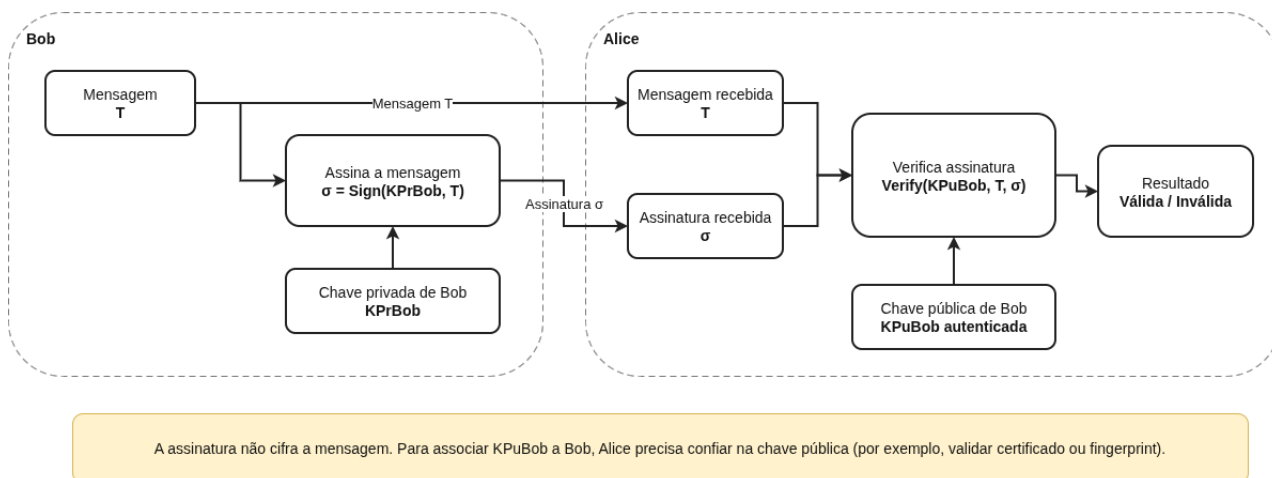


Figura 6: Bob assina uma mensagem e Alice verifica a assinatura com a chave pública autenticada

1. **Criação do par de chaves:** Bob gera uma chave privada RSA e extrai dela a chave pública correspondente. A chave privada deve permanecer sob o controle de Bob; a chave pública deve ser autenticada antes de Alice confiar nela.

```
$ umask 077
$ openssl genpkey -algorithm RSA -out KPrBob.pem -pkeyopt rsa_keygen_bits:3072
$ openssl pkey -in KPrBob.pem -pubout -out KPubBob.pem
```

2. **Assinatura da mensagem:** Bob assina os bytes de `arquivo.txt` com RSA-PSS e SHA-256. Ele envia o arquivo original e `signature.bin` para Alice.

```
$ openssl dgst -sha256 -sign KPrBob.pem \
-sigopt rsa_padding_mode:pss \
-sigopt rsa_pss_saltlen:digest \
-out signature.bin arquivo.txt
```

3. **Verificação da assinatura:** Alice verifica a assinatura sobre a cópia recebida de `arquivo.txt`, usando a chave pública autenticada de Bob e os mesmos parâmetros RSA-PSS.

```
$ openssl dgst -sha256 -verify KPubBob.pem \
-sigopt rsa_padding_mode:pss \
-sigopt rsa_pss_saltlen:digest \
-signature signature.bin arquivo.txt
Verified OK
```

Uma verificação bem-sucedida indica que a assinatura corresponde ao arquivo e à chave pública fornecidos. Se a mensagem ou a assinatura tiver sido alterada, ou se for usada





outra chave pública, a verificação falha.

O que a assinatura garante

- **Integridade:** permite detectar alterações nos dados assinados.
- **Autenticidade da chave:** confirma que a assinatura corresponde à chave privada associada à chave pública usada; associar essa chave a Bob exige autenticar a chave pública.
- **Sem confidencialidade:** qualquer pessoa que obtenha a mensagem pode lê-la; para ocultar seu conteúdo, é necessário cifrá-la separadamente.





6 Confidencialidade com Criptografia Híbrida

Na criptografia híbrida, Bob gera uma chave de dados aleatória (DEK) e usa uma cifra simétrica para proteger a mensagem. Em seguida, cifra a DEK com a chave pública de Alice. Neste exemplo, $M1$ é a mensagem cifrada com AES-256-CBC e $M2$ é a DEK cifrada com RSA-OAEP e SHA-256. Bob envia $M1$, $M2$ e o IV de $M1$; Alice recupera a DEK com sua chave privada e então decifra $M1$. O IV não é secreto, mas precisa acompanhar $M1$. O servidor pode armazenar e encaminhar esses valores, mas não consegue recuperar a mensagem sem a chave privada de Alice.

Bob precisa obter a chave pública de Alice por um meio confiável e validar que ela realmente pertence a Alice, por exemplo conferindo um certificado ou um fingerprint por canal independente. Cifrar a DEK com RSA-OAEP protege essa chave para Alice, mas não autentica Bob.

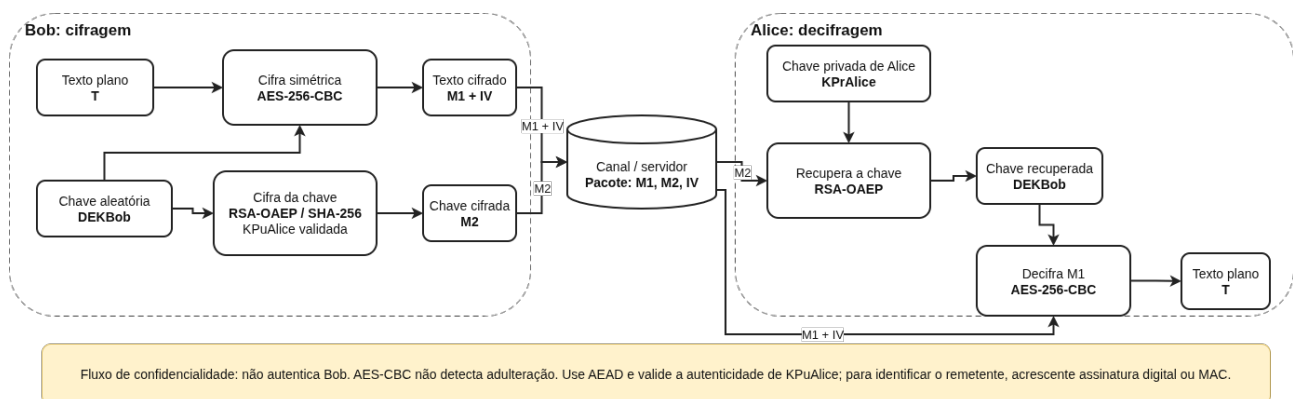


Figura 7: Proteção híbrida da mensagem: AES cifra os dados e RSA-OAEP protege a chave de dados

Criação do par de chaves de Alice

Alice cria um par RSA. Ela mantém $KPrAlice$ em segredo e disponibiliza $KPAlice$ com um mecanismo que permita a Bob autenticar a chave pública.

```
$ umask 077
$ openssl genpkey -algorithm RSA -out KPrAlice.pem -pkeyopt rsa_keygen_bits:3072
$ openssl pkey -in KPrAlice.pem -pubout -out KPAlice.pem
```

Cifragem híbrida (Bob)

Bob gera uma DEK aleatória de 256 bits, cifra $T.txt$ com AES-256-CBC e cifra a DEK com $KPAlice$ usando RSA-OAEP/SHA-256.

1. Gerar a DEK e cifrar o texto. $M1.iv$ deve acompanhar $M1.enc$.





```
$ umask 077
$ openssl rand -out DEKBob.bin 32
$ DEK_HEX=$(od -An -v -tx1 DEKBob.bin | tr -d ' \n')
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M1.iv
$ openssl enc -aes-256-cbc -K "$DEK_HEX" -iv "$IV_HEX" \
  -in T.txt -out M1.enc
```

2. Cifrar a DEK com a chave pública autenticada de Alice, gerando M2.

```
$ openssl pkeyutl -encrypt -in DEKBob.bin -inkey KPuAlice.pem -pubin \
  -out M2.enc -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256
```

3. Enviar M1.enc, M1.iv e M2.enc para Alice ou armazená-los no servidor para ela recuperar depois.

Decifragem (Alice)

Alice usa KPrAlice para recuperar a DEK de M2 e, com essa DEK e o IV, decifra M1.

1. Recuperar a DEK de M2.

```
$ openssl pkeyutl -decrypt -in M2.enc -inkey KPrAlice.pem \
  -out DEKBob.bin -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256
```

2. Decifrar M1 com a DEK recuperada e o IV associado.

```
$ DEK_HEX=$(od -An -v -tx1 DEKBob.bin | tr -d ' \n')
$ openssl enc -d -aes-256-cbc -K "$DEK_HEX" -iv "$(cat M1.iv)" \
  -in M1.enc -out T.txt
```

Limites deste exemplo

AES-CBC fornece confidencialidade, mas não integridade: alterações em *M1* ou no IV podem não ser detectadas e podem modificar o texto decifrado. O fluxo também não prova que Bob enviou a mensagem, e RSA-OAEP não autentica o remetente nem vincula o IV a *M1*. Em produção, use um protocolo híbrido estabelecido que autentique o conteúdo e seus parâmetros, por exemplo com uma cifra AEAD como AES-GCM, garantindo um nonce único por chave; autentique Bob com uma assinatura digital verificada por uma chave pública confiável ou por um protocolo autenticado apropriado. A interface `openssl enc` usada neste exemplo não oferece cifras AEAD como AES-GCM, então não basta trocar o nome da cifra nos comandos. Os comandos com `openssl enc -K` são didáticos: a DEK aparece nos argumentos do processo e não deve ser manipulada dessa forma em produção.





Copyright

All rights reserved to MT4 Tecnologia LTDA.

Autor

Caio Abreu Ferreira [cferreiras@senhasegura.com]

MT4 Tecnologia Ltda.

Rua Joaquim Antunes, 767 – Pinheiros – São Paulo

Phone: +55 11 3065-7070

www.mt4.com.br

support@senhasegura.com

